Combinator Calculus: Decidable Type Checking in Extensional Type Theory

Lexi Aiello[1] and Eric Klavins[2]

[1]Department of Philosophy, University of Washington
[2]Electrical & Computer Engineering Department, University of Washington

August 10, 2026

Abstract

TODO

1 Introduction

TODO later after main sections done

1.1 Results

- In 2, we present our extension of the categorical combinators [3] [4] to dependent types. We have no variables, and no contexts. Our extension is analagous in expressivity to the Calculus of Constructions on the lambda cube [2]. We formally present our β-reduction rules, inference rules, and η-reduction rules.
- In 4.5, we prove that computable function extensionality is a natural consequence of η-reduction in the calculus. Our proof is formalized as a metatheorem in Lean 4.
- In 5.1, we prove that equality reflection is a natural consequence of our design.
- In 5, we prove that type-checking is decidable.
- In *TODO insert here*, we present a procedure for compiling a small subset of Lean 4 (base Calculus of Constructions) to DCC.

All of our results have been formalized in the Lean 4 proof assistant. To this date, we are not aware of any calculus where these features can coexist: extensional equality elimination; computable function extensionality; and decidable type-checking. However, due to our choice of affine logic and categorical combinators, we inherit them by construction. We do not present a combinatory basis for inductive types, as they are out of scope for this paper. We do not assume equality reflection or function extensionality as axioms. They are computable definitions, not theorems in Prop.

1.2 Background

1.3 The SK Combinator Calculus: not quite enough

Prior iterations of the calculus, and attempts by others at combinatory dependent type theory have used the SK combinator calculus [1, p. 10:14].

$$Kxy \coloneqq x$$
$$Sxyz \coloneqq (xz)(yz)$$

Figure 1: Rewrite rules of the SK combinator calculus

$$
\begin{aligned}
K &: (A : \text{Sort } u) \to (B : A \to \text{Sort } v) \to (x : A) \to Bx \to A \\
Sxyz &: (A : \text{Sort } u) \to (B : A \to \text{Sort } v) \to (C : (x : A) \to Bx \to \text{Sort } s) \\
&\to (x : A) \to (y : Bx) \to (z : A) \to Cz(yz)
\end{aligned}
$$

Figure 2: The usual dependent types of the S and K combinators

$$
\begin{aligned}
\text{Fst}(x, y) &\coloneqq x \\
\text{Snd}(x, y) &\coloneqq x \\
(x \circ y)z &\coloneqq x(yz) \\
\text{Id}x &\coloneqq x \\
\langle x, y\rangle x &\coloneqq (Mx, Nx) \\
\text{App}(x, y) &\coloneqq xy \\
\Lambda(x)yz &\coloneqq x(y, z)
\end{aligned}
$$

Figure 3: Reduction rules of the categorical combinators

However, past attempts posed severe limitations due to minimalism of SK and its lack of structures analagous to Π types. S and K are the only primtive objects. Π types are usually supported by a Π combinator. Extra helper ("meta") combinators are usually required to mechanically suffle terms in the context under Π binders [1, 10:17].

The usual dependent types of S and K given by 2 [1, p. 10:16].

SK's minimalism results in prohibitively large [1, 10:17] types, and a great burden on the prover. Complex machinery is required to shuffle elements in the context around (through combinations of S and K). Universe levels must be provided explicitly, which are very difficult for a user to calculate. In general, types are almost impossible for a user to write [1, p. 10:17]. Also, the kernel cannot reason about structural similarities between types, resulting in definitional equality errors. Non-termination is intrinsic to SK, since S duplicates terms. Our past naive solution used the $BCKI$ calculus, with the hope that types would be easier to write and normalization inherent. Still, $BCKI$ does not use term points, resulting in the same practical limitations of SK.

DCC is based on the categorical combinator calculus. In DCC, all universe levels are inferred except in Sort u terms. Types arguments are internalized. Types are much easier for the user to express. This is due to the isomoprhism between currying in the categorical combinators and λ-abstraction. Debruijn indices are modeled by nested tuples (an "environment", or a "couple") and projections, substitution by tuple currying, and application by evaluation (3). Types are also much smaller due to this similarity, and the kernel can reason about structural equivalence more often (even without η-reduction).

1.4 Categorical Combinators

The rewrite rules of the categorical combinator calculus are given by 3 [3].

Each combinator corresponds to an operation in the cartesian closed category. Id corresponds to the identity morphism, composition to morphism composition, App to evaluation, and Λ to currying.

Constants can be encoded without a constant combinator (as in SK) 4.

An isomorphism exists between the categorical combinators and the λ-calculus. For example, the compilation of λ-calculus with Debruijn indices to categorical combinators is quite simple [3, p. 4]:

$$
\begin{aligned}
[\![\lambda.M]\!] &\coloneqq \Lambda([\![M]\!]) \\
[\![0]\!] &\coloneqq \text{Snd} \\
[\![n+1]\!] &\coloneqq [\![n]\!] \circ \text{Fst} \\
[\![MN]\!] &\coloneqq App \circ \langle [\![M]\!], [\![N]\!] \rangle
\end{aligned}
$$

The first top-level λ-application which is not under a binder is applied with the terminal object $\mathbb{1}$:

$$
\begin{aligned}
`(c) &\coloneqq \Lambda(\mathrm{Fst})c \\
`(c)x &= \Lambda(\mathrm{Fst})cx \\
&= \mathrm{Fst}(c, x) \\
&= c
\end{aligned}
$$

Figure 4: Constants are derived

$$
\textsc{Fst}\ \frac{}{\mathrm{Fst}(A, B) : A \times B \to A} \qquad \textsc{Snd}\ \frac{}{\mathrm{Snd}(A, B) : A \times B \to B} \qquad \textsc{Id}\ \frac{}{Id(A) : A \to A}
$$

$$
\textsc{App}\ \frac{}{\mathrm{App}(A, B) : ((A \to B) \times A) \to B} \qquad \textsc{Curry}\ \frac{x : A \times B \to C}{\Lambda(A, B, x) : A \to B \to C}
$$

$$
\textsc{Pair}\ \frac{x : A \to B, y : A \to C}{\langle x, y \rangle : A \to B \times C} \qquad \textsc{Comp}\ \frac{x : B \to C, y : A \to B}{x \circ y : A \to C}
$$

Figure 5: Inference rules of the simply-typed categorical combinators

$$
[\![(\lambda M)N]\!] \coloneqq [\![M]\!]\mathbb{1}[\![N]\!] \tag{1}
$$

For example:

$$
\begin{aligned}
(\lambda 0)c &= \Lambda(\mathrm{Snd})\mathbb{1}c \\
&= \mathrm{Snd}(\mathbb{1}, c) \\
&= c
\end{aligned}
$$

Compared to the SK combinator calculus, writing programs with categorical combinators feels quite natural. However, adapting it to dependent types has not been attempted thus far.

1.5 The Simply-Typed Categorical Combinators

A simply-typed variant of the categorical combinators is presented in [4, p. 220]. Type arguments are given as points, rather than applied arguments as in dependently-typed SK. Type arguments can be inferred altogether in Λ, App, and Pair 5.

2 DCC: The Dependent Combinator Calculus

We present DCC: the dependent combinator calculus. DCC is extensional with computable function extensionality and equality reflection as metatheorems. It is based on an adaptation of the categorical combinators to dependent types and affine logic. It is equivalent to the base Calculus of Constructions in expressivity with respect to the λ-cube.

We believe our choice of combining categorical combinators with dependent types and affine logic is uniquely suitable for decidable type-checking in extensional type theory. To date, this calculus is the only one of its kind. Dependently-typed combinators have been largely deemed highly nontrivial. Decidable type-checking in extensional type theory has thus far been thought to be impossible [5, p. 2].

We achieve computable function extensionality through the observation that in combinatory logic, open terms do not exist, and canonical terms in η-reduction correspond to the primitive operations in cartesian closed categories. η-reductions are deterministic, unique, and mutually exclusive in behavior. Hence, the set of canonical functions is small and well-behaved. Function extensionality is provable by demonstrating that if two η-reduced terms f and g are applied the same value, and the value is not contained in the respective functions, then $fx =_\beta gx$ iff their η-reductions are the same. That is, $f = g$. We elaborate more on this proof technique later.

We achieve dependent typing by making type arguments functions of the environment as term points, rather than as arguments or as static values. To give an analogy, the categorical combinators correspond to cartesian closed categories, while DCC corresponds to locally cartesian closed categories. Locally cartesian closed categories are known to be a semantics for dependent type theory[8].

2.1 β-Reduction Rules

DCC has the same reduction rules as in the categorical combinator calculus, except Π types are added, products become Σ-types, the pair rule is restricted affinely, some basic operations on affine terms are supported, and a sort hierarchy is added. With minimal modifications, the categorical combinator calculus becomes a highly expressive type theory.

Affine logic is supported through "box" terms, whose boxed values are points:

$$\frac{\textsc{Box}\quad x : A}{\&(x) : !(A)}$$

Boxes are consumed by pairs, thus taming the sole source of duplication in the calculus and yielding strong normalization by a syntactic decreasing measure.:

$$\frac{\textsc{Pair}}{[\![B]\!]\langle x, y\rangle \&(z) \rightarrow_\beta [\![B]\!](Mx, Nx(Mx))} \tag{2}$$

Figure 6: Pairs consume boxes, and decrease the size of the program while duplicating a value.

The full reduction rules are given in 7.

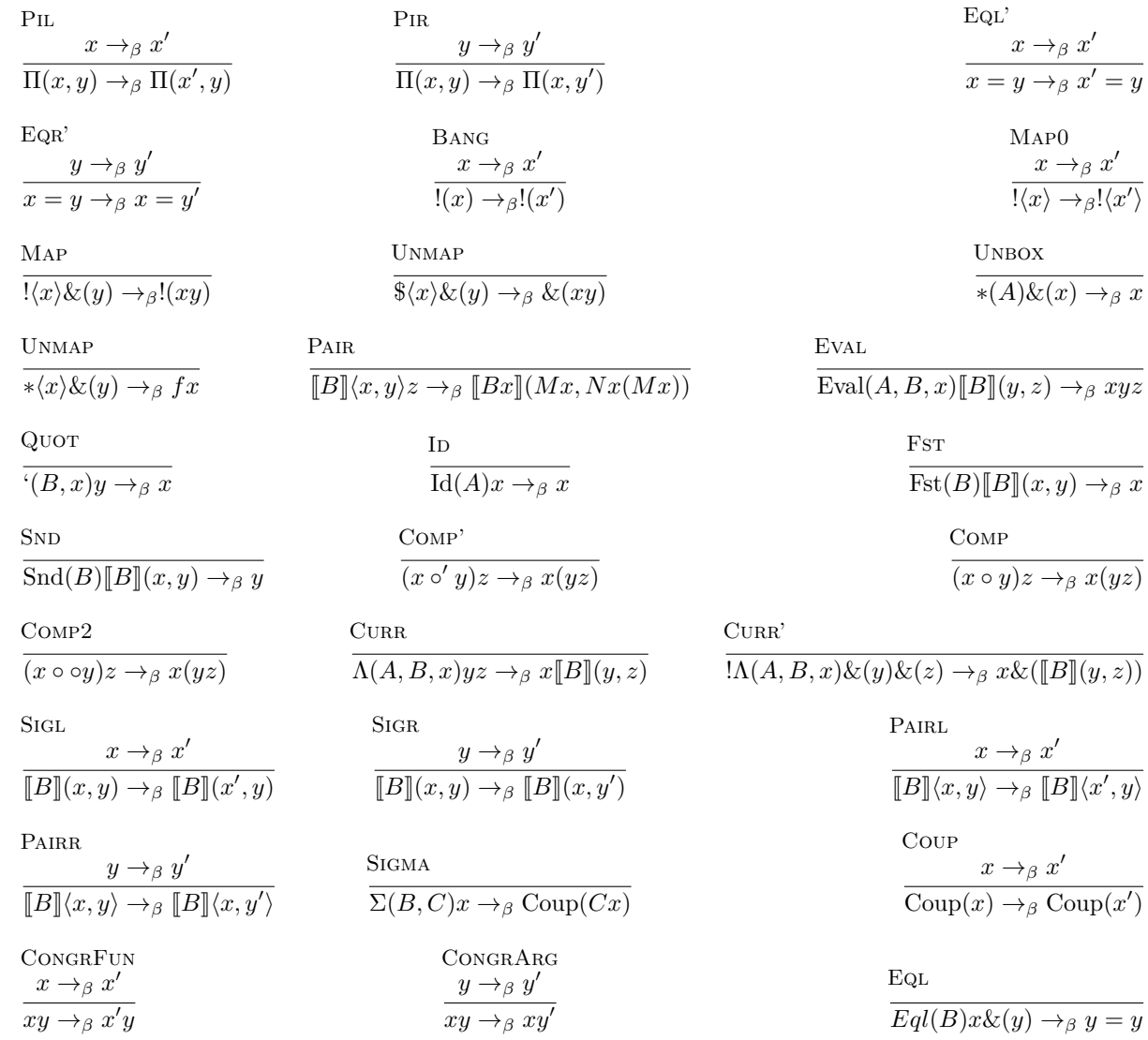


Figure 7: Where $\to_\beta$ represents single-step beta-reduction. Unmap is not strictly necessary, but is included for performance optimization purposes in compiled code.

☑ [Formalized: `Dcc.GenericExpr.RedAff`]

Since DCC is strongly normalizing by default, conversion in the judgment relation is avoided altogether. Types are eagerly reduced in every inference rule, instead of being permitted by general catch-all conversion. As a consequence, there is exactly one valid judgment for a given term. This had wide, sweeping ramifications, enabling equality reflection, and computable function extensionality when paired with η-reduction, while retaining decidability. As in intensional type theory, refl is the only inhabitant of equality. Since conversion is avoided, $x = x$ is the *only* valid equality judgment. Induction on conversion is also conveniently avoided in metatheorems due to eager type normalization.

2.2 Types as terms: Environments are contexts

In DCC, combinator types are determined by points which index an environment and eliminate into a Sort u. This approach has the advantage of requiring no explicit universe levels (whereas dependent SK does), and type arguments can often be inferred, easing the burden on the user.

This design decision enables a natural compilation from Lean 4 to DCC. Interpreting couple environments as Σ-types yields a telescope context:

$$x_0 : A_0, x_1 : A_1(x_0), x_2 : A_2(x_0, x_1) \approx [\![\Sigma(A_1), A_2]\!]([\![A_1]\!](x_0, x_1), x_2) \tag{3}$$

Figure 8: Couple environments are analogous to context substitutions

Dependent types are supported in principle by a Π combinator. Unlike in prior attempts with SK, its domain is a point and a function of an environment and its codomain of the environment and corresponding term argument:

$$\textsc{Pi}\ \frac{B : A \to \text{Sort } u, C : (x : A) \to (y : Bx) \to \text{Sort } v}{\Pi(B, C) : (x : A) \to (y : Bx) \to \text{Sort } max(u, v)} \qquad \textsc{App}\ \frac{f : (\Pi(B, C)\text{env}), B \text{ env} \to^!_\beta A, x : A, C \text{ env } x \to^!_\beta D}{fx : D}$$

Π types model move semantics by threading terms in the context through nested Π terms when type-checking an application. Duplication is avoided, unlike in approaches where substitution is recursive.

Arrows are defined as Π terms where $x : A$, $A : \text{Sort } u$, $B \coloneqq \text{Id}(\text{Sort } u)$, and $C \coloneqq \text{`}(\text{Sort } u, \text{`}(A, C'))$. For example, the type of $\text{Id}(A)$ is written literally like such:

$$\textsc{Id}\ \frac{A : \text{Sort } u, B : \text{Sort } v}{\text{Id}(A) : (\Pi(\text{Id}(\text{Sort } u), \text{`}(\text{Sort } u, \text{`}(A, B)))A)} \tag{4}$$

2.3 Inference Rules

We make use of the standard notation for Π types and Σ types in our illustration of our inference rules. However, the full types are available in the appendix. Our full inference rules are given in 9.

```
def Sigma.curry {γ : ∀ a, β a → Type*} (f
↪  : ∀ x : Sigma β, γ x.1 x.2) (x : α)
↪  (y : β x) : γ x y :=
  f ⟨x, y⟩
```

$$\textsc{Curr}\ \frac{B : A \to \mathrm{Sort}\ u, x : (x : (x : A) \times Bx) \to Cx}{\Lambda(A, B, x) : (x : A) \to (y : Bx) \to C[\![B]\!](x, y)} \quad (5)$$

Figure 10: Λ behaves identically to dependent currying in Mathlib in Lean 4.

```
def Sigma.uncurry {γ : ∀ a, β a → Type*}
↪  (f : ∀ (x) (y : β x), γ x y) (x :
↪  Sigma β) : γ x.1 x.2 :=
  f x.1 x.2
```

$$\textsc{Eval}\ \frac{A : \mathrm{Sort}\ u, B : A \to \mathrm{Sort}\ v, x : (x : A) \to (y : Bx) \to Cxy}{\mathrm{Eval}(A, B, x) : (x : (x : A) \times Bx) \to \mathrm{Eval}(A, B, C)} \quad (6)$$

Figure 11: Eval behaves identically to dependent uncurrying in Mathlib in Lean 4.

$$\textsc{Eq}\ \frac{x : A, y : A}{(x = y) : \mathrm{Sort}\ 0} \qquad \textsc{Id}\ \frac{A : \mathrm{Sort}\ u}{\mathrm{Id}(A) : A \to A} \qquad \textsc{Sort}\ \frac{}{\mathrm{Sort}\ u : \mathrm{Sort}\ u + 1} \qquad \textsc{Top}\ \frac{}{\top : \mathrm{Sort}\ 0} \qquad \textsc{Bot}\ \frac{}{\bot : \mathrm{Sort}\ 0} \qquad \textsc{Tt}\ \frac{}{* : \top}$$

$$\textsc{Refl}\ \frac{B : A \to \mathrm{Sort}\ u}{\mathrm{refl}(B) : (x :!(A)) \to (y : (!\langle B\rangle x)) \to (\mathrm{Eql}(B)xx)} \qquad \textsc{Eql}\ \frac{B : A \to \mathrm{Sort}\ u}{\mathrm{eql}(B) : (x :!(A)) \to (y :!\langle B\rangle x) \to \mathrm{Sort}\ 0}$$

$$\textsc{Comp}\ \frac{g : (x :!(A)) \to ((y : A) \times By), f : (x : (x : A) \times Bx) \to Cx}{f \circ g : (x :!(A)) \to (C \circ g)x} \qquad \textsc{Comp'}\ \frac{g : A \to B, f : (x : B) \to Cx}{f \circ' g : (x : A) \to (C \circ' g)x}$$

$$\textsc{Comp2}\ \frac{g : (x :!(A)) \to ((y : A) \times Bx), f : (x : (y : A) \times Bx) \to Cx}{f \circ\circ g : (x :!(A)) \to (C \circ\circ g)x} \qquad \textsc{Coup}\ \frac{A : \mathrm{Sort}\ u, B : A \to \mathrm{Sort}\ v}{\mathrm{Coup}(B) : \mathrm{Sort}\ max(u, v)}$$

$$\textsc{Fst}\ \frac{B : A \to \mathrm{Sort}\ v}{\mathrm{Fst}(B) : (x : A) \times Bx \to A} \qquad \textsc{Snd}\ \frac{B : A \to \mathrm{Sort}\ v}{\mathrm{Snd}(B) : (x : (x : A) \times Bx) \to B(\mathrm{Fst}(B)x)}$$

$$\textsc{Sigma}\ \frac{B : A \to \mathrm{Sort}\ u, C : (x : A) \to (y : Bx) \to \mathrm{Sort}\ v}{\Sigma(B, C) : A \to \mathrm{Sort}\ max(u, v)} \qquad \textsc{Mkc}\ \frac{x : A, y : Bx}{[\![B]\!](x, y) : (x : A) \times Bx}$$

$$\textsc{Curr}\ \frac{B : A \to \mathrm{Sort}\ u, x : (x : (x : A) \times Bx) \to Cx}{\Lambda(A, B, x) : (x : A) \to (y : Bx) \to C[\![B]\!](x, y)} \qquad \textsc{Map}\ \frac{x : A \to \mathrm{Sort}\ u}{!\langle x\rangle :!(A) \to \mathrm{Sort}\ n}$$

$$\textsc{Curr'}\ \frac{B : A \to \mathrm{Sort}\ u, x :!((x : A) \times Bx)* \to \langle C\rangle x}{!\Lambda(A, B, x) : (\&(x) :!(A)) \to (\&(y) : \langle B\rangle x) \to C\&([\![B]\!](x, y))} \qquad \textsc{Fmap}\ \frac{x : (x : A) \to Bx}{\$\langle x\rangle : (\&(x) :!(A)) \to !(Bx)}$$

$$\textsc{Eval}\ \frac{A : \mathrm{Sort}\ u, B : A \to \mathrm{Sort}\ v, x : (x : A) \to (y : Bx) \to Cxy}{\mathrm{Eval}(A, B, x) : (x : (x : A) \times Bx) \to \mathrm{Eval}(A, B, C)} \qquad \textsc{Quot}\ \frac{x : A, B : \mathrm{Sort}\ u}{`(B, x) : B \to A}$$

$$\textsc{Pair}\ \frac{x : (x : A) \to Bx, y : (x : A) \to (y : Bx) \to Cxy}{[\![C]\!]\langle x, y\rangle : (x : A) \to (y : Bx) \times Cxy} \qquad \textsc{Pi}\ \frac{B : A \to \mathrm{Sort}\ u, C : (x : A) \to (y : Bx) \to \mathrm{Sort}\ v}{\Pi(B, C) : (x : A) \to (y : Bx) \to \mathrm{Sort}\ max(u, v)}$$

$$\textsc{Bang}\ \frac{A : \mathrm{Sort}\ u}{!(A) : \mathrm{Sort}\ u} \qquad \textsc{Box}\ \frac{x : A}{\&(x) :!(A)} \qquad \textsc{Unmap}\ \frac{x : (x : A) \to Bx}{*\langle x\rangle : (x :!(A)) \to *\langle B\rangle} \qquad \textsc{Unbox}\ \frac{A : \mathrm{Sort}\ u}{*(A) :!(A) \to A} \qquad \textsc{App}\ \frac{f : (x : A_1) \to Bx, x : A_2, A_1 \equiv A_2}{fx : Bx} \qquad \textsc{Def. Eq.}\ \frac{A_1 \to^!_\beta B_1, B_1 \to^!_\eta C \qquad A_2 \to^!_\beta B_2, B_2 \to^!_\eta C}{A_1 \equiv A_2}$$

Figure 9: $(x : A) \times Bx$ is notation for $\mathrm{Coup}(B)$. $(x : A) \to Bx$ is notation for $\Pi(\mathrm{Id}(\mathrm{Sort}\ m), `(\mathrm{Sort}\ m, B))A$, where $A : \mathrm{Sort}\ m$. The above inference rules implicitly β-η reduce the inferred type to normal form. This has been omitted for legibility.

☑ [Formalized: `Dcc.GenericExpr.Judge3`]

Our inference rules are intuitive. For example, curry and eval mirror Σ currying and uncurrying in Lean 4 [6]:

Our usage of points as indexes of the environment is appealing, since universe levels are never specified explicitly, thus avoiding universe arithmetic.

2.3.1 Examples of Type-Checking

The dependent K combinator is written in Lean as:

```
def K (u v : ℕ) (A : Sort u) (B : A → Sort v) (x : A) (y : B x) : A := x
```

```
#check K 3 2 (Sort 2) (fun x => x) (Sort 1) (Sort 0)
/-
K 3 2 (Sort 2) (fun x => x) (Sort 1) (Sort 0) : Type 1
-/
```

Its compiled DCC expression is available in the appendix A.1. It type-checks like such:

```
def K_example₃' : Expr :=
  dcc_infer' ((fun (α : Sort 3) (β : α → Sort 2) (x : α) (_y : β x) => x) (Sort 2) (fun x =>
  ↪  x) (Sort 1) (Sort 0))
/-
Messages here:
33:0:
(some Sort 2)
-/
```

Notably, equalities between functions are provable with `refl` in DCC due to computable function extensionality:

```
#eval (DCC (Id(Id(Sort 0) = Id(Sort 0) ∘' Id(Sort 0) ∘' Id(Sort 0))
  (refl(Id(Sort 1)) &((Π(Id(Sort 1), `(Sort 1, `(Sort 0, Sort 0))) Sort 0))
                    &(Id(Sort 0))))).infer.isOk
/-
Messages here:
195:0:
true
-/
```

Here, `Id(Id(Sort 0) = Id(Sort 0) ∘' Id(Sort 0) ∘' Id(Sort 0))` expects a proof of an equality between two functions which are not $=_\beta$. However, `refl` suffices due to eager β-η-reduction (specifically well-behaved η-reduction) and judgment canonicity. ☑ [Formalized: `Dcc.Examples3`]

3 Affine logic: strong normalization for free

Contraction is restricted in DCC, as in affine logic. Strong normalization is inherited as such and provable by a simple syntactic decreasing measure. Confluence holds in DCC, so we inherit global confluence (by Newmann's lemma), and decidability of $=_\beta$. The size measure sums the size of each combinator's points and performs simple arithmetic for an ordering matching combinator precedence. In boxes, the size of its point x is counted thrice so that consuming it with a pair decreases the program's size, rather than increasing it:

Theorem 1. *If A and B are terms and $A \to_\beta B$, then $size(B) < size(A)$.*

Proof. By induction on the reduction relation.

```
theorem RedAff.decreasing' (h : RedAff A B) : size B < size A := by
  induction h <;> grind
```

☑ [Formalized: `Dcc.GenericExpr.RedAff.decreasing'`] □

3.1 Strong Normalization

Theorem 2. *If A is a term, and if that $A \to_\beta B$ implies B is accessible forall terms B, then A is accessible.*

Proof. Suppose A and B are terms such that $A \to_\beta B$ and B is accessible. Then, by well-founded induction with respect to the size of A (by), A is accessible.

```
@[dcc_force_reduce, dcc_force_aff]
instance RedAff.instRedRel.{u} {τ : Type u} : RedRel (α := GenericExpr τ) (RedAff (τ := τ))
↪  size where
  run := step?
  decreasing := decreasing'

def RedRel.sn.{u} {α : Type u} {r : α → α → Prop} (t : outParam (α → ℕ))
  {m : Sort ((max u 0) + 1) → Sort ((max u 0) + 1)} [Monad m]
```

```
@[aesop norm (rule_sets := [DccSize]), dcc_size, grind]
def size.{u} {τ : Type u} (e : GenericExpr τ) : ℕ :=
  match e with
  | DCC[.] ⋆ => 1
  | DCC[.] ⊤ => 2
  | DCC[.] ⊥ => 2
  | DCC[.] x = y => (x.size + y.size) + 1
  | DCC[.] Eql(α)  => α.size + 5
  | DCC[.] refl(α) => α.size + 5
  | DCC[.] Sort m => m
  | DCC[.] Σ(α, β) => α.size + β.size + 3
  | DCC[.] `(β, x) => β.size + x.size + 3
  | DCC[.] ⟦_β⟧(x, y) => x.size + y.size
  | DCC[.] Coup(β) => β.size + 1
  | DCC[.] Id(α)      => 1 + α.size
  | DCC[.] Fst(α)    => 3 + α.size
  | DCC[.] Snd(α)    => 3 + α.size
  | DCC[.] Π(α, β) => α.size + β.size + 2
  | DCC[.] (f ∘' g) => f.size + f.size + g.size + 3
  | DCC[.] (f ∘ g)  => f.size + f.size + g.size + 3
  | DCC[.] (f ∘∘ g)  => f.size + f.size + g.size + 3
  | DCC[.] ⟦_β⟧⟨a, b⟩   => a.size + a.size + b.size + 3
  | DCC[.] Λ(α, β, x) => α.size + β.size + x.size + 3
  | DCC[.] !Λ(α, β, x) => α.size + β.size + x.size + 3
  | DCC[.] Eval(α, β, x) => α.size + β.size + x.size + 2
  | DCC[.] !(x)   => x.size + 1
  | DCC[.] *(α)   => α.size + 3
  | DCC[.] *<α>   => α.size + 1
  | DCC[.] &(x)   => x.size + x.size + x.size + x.size + x.size
  | DCC[.] !<x>   => x.size + x.size + x.size + 3
  | DCC[.] $<x>   => x.size + x.size + x.size + x.size + 3
  | DCC[.] (f x) => f.size + x.size
  | .const _ => 0
```

Figure 12: Our basic syntactic term size measure: ☑ [Formalized: `Dcc.GenericExpr.size`]

```
  (hR : RedRel r t (m := m)) (e : α) : Acc (fun b a => r a b) e := Acc.intro e (fun e' hr =>
  ↪  by
    have h := hR.decreasing hr
    exact sn t hR e')
termination_by t e
```

☑ [Formalized: `Dcc.GenericExpr.RedRel.sn`] □

3.2 Confluence

Local and global confluence hold in DCC. Our step relation uses a call-by-value semantics, so reduction paths are not deterministic. It is provable by parallel induction on $A \to_\beta B$ and case-splitting on $A \to_\beta C$.

Theorem 3. *If A, B, and C are terms and $A \to_\beta B$ and $A \to_\beta C$, then there exists D such that $B \to_\beta^* D$ and $C \to_\beta^D$.*

Proof. By parallel induction on $A \to_\beta B$ and case-splitting on $A \to_\beta C$.

```
theorem RedAff.conf.{u} {α : Type u} {A B C : GenericExpr α} (h₁ : (RedAff (τ := α)) A B) (h₂
↪  : RedAff A C) : ∃ D, RedAff.instRedRel.ReflTrans B D ∧ RedAff.instRedRel.ReflTrans C D :=
↪  by
  induction h₁ generalizing C with
  | evalc =>
    cases h₂ with
    | evalc => exact ⟨_, .refl, .refl⟩
    | congrFun h => cases h
    | congrArg h =>
      cases h with
      | sigl h => exact ⟨_, .single (.congrFun (.congrArg h)), .single .evalc⟩
      | sigr h => exact ⟨_, .single (.congrArg h), .single .evalc⟩
  | curr =>
    cases h₂ with
```

```
    | curr => exact ⟨_, .refl, .refl⟩
    | congrFun h =>
      cases h with
      | congrFun h => cases h
      | congrArg h => exact ⟨_, .single (.congrArg (.sigl h)), .single .curr⟩
    | congrArg h => exact ⟨_, .single (.congrArg (.sigr (by assumption))), .single .curr⟩
  | congrFun h_r ih =>
    cases h₂
    case comp₂ | fmap | unmap | unbox | fst | snd | sigma | evalc | comp' | quot | id | comp
    ↪  => cases h_r
    case _ => sorry -- omitted for brevity
    case congrFun f f' x f'' h =>
      have ⟨_, h₁, h₂⟩ := ih h
      exact ⟨_, congrFun' h₁, congrFun' h₂⟩
    case congrArg f f' x x'' h_x =>
      exact ⟨_, congrArg' (.single h_x), congrFun' (.single h_r)⟩
  | congrArg h_r ih =>
    cases h₂ with
    | eql
    | fmap
    | map | unbox | unmap
    | pair => cases h_r
    | evalc => cases h_r with
      | sigl h => exact ⟨_, .single .evalc, .single (.congrFun (.congrArg h))⟩
      | sigr h => exact ⟨_, .single .evalc, .single (.congrArg h)⟩
    | _ => sorry -- omitted for brevity
    | congrFun => exact ⟨_, congrFun' (.single (by assumption)), congrArg' (.single (by
    ↪  assumption))⟩
    | congrArg h =>
      have ⟨D, h₁, h₂⟩ := ih h
      exact ⟨_, congrArg' h₁, congrArg' h₂⟩
  | _ => sorry -- omitted for brevity
```

We have omitted most of the proof for brevity. ☑ [Formalized: `Dcc.GenericExpr.RedAff.conf`] □

Theorem 4. *If A, B, and C are terms and $A \to_\beta^* B$ and $A \to_\beta^* C$, then there exists D such that $B \to_\beta^* D$ and $C \to_\beta^D$.*

Proof. By Newmann's lemma [7] and 3:

Suppose A, B, and C are terms, and $h_1 : A \to_\beta^* B$ and $h_2 : A \to_\beta^* C$. We proceed by head induction on h_1 with respect to the reduction relation.

- Refl case: Suppose $h_1 = \text{refl}$. Then, $A = B$. We continue by case-splitting on h_2:
  - Refl case: Suppose $h_2 = \text{refl}$. Then, $B = C$. So, $A = B$ and $A = C$. Therefore, $\text{refl} : B \to_\beta^* A$ and $\text{refl} : C \to_\beta^* A$, so there exists D such that $B \to_\beta^* D$ and $C \to_\beta^* D$.
  - Step case: Suppose there exists $hr : A \to_\beta C$ and $h_2 = \text{single}hr$. Then, $B \to_\beta C$ and $C \to_\beta^* C$. By the inductive hypothesis, there exists D such that $B \to_\beta^* D$ and $C \to_\beta^* D$.
- Step case: Suppose there exists $hr : A \to_\beta B$. We continue by case-splitting on h_2:
  - Refl case: Suppose $h_2 = \text{refl}$. Then, $B = C$, so $\text{refl} : B \to_\beta^* C$ and $\text{refl} : C \to_\beta^* C$. Hence, there exists D such that $B \to_\beta^* D$ and $C \to_\beta^* D$.
  - Step case: Suppose there exists hr_2 such that $hr_2 : A \to_\beta C$. Then, by local confluence, there exists D such that $B \to_\beta^* D$ and $C \to_\beta^* D$.

  ☑ [Formalized: `Dcc.GenericExpr.Relation.RedRell.globally_confluent`] □

3.3 Uniqueness of normal forms

Theorem 5. *If A and B are terms, then, forall C where C is a term and $A \to_\beta^* B$ and $A \to_\beta^* C$ where B and C are normal forms, then $B = C$.*

Proof. Suppose A, B, and C are terms, and $hr_1 : A \to_\beta^* B$ and $hr_2 : A \to_\beta^* C$ such that B and C are normal forms. Then, by global confluence 4, there exists D, hr_3, and hr_4 such that $hr_3 : B \to_\beta^* D$ and $hr_4 : C \to_\beta^* D$. We proceed by parallel case-splitting on hr_3 and hr_4 and well-founded induction with respect to term size 3.1 on the step relation.

- Refl, refl case: Suppose $hr_1 = \text{refl}$ and $hr_2 = \text{refl}$. Then, $A = B$ and $A = C$, so $B = C$.
- Step, refl case: Suppose there exists hr such that $hr_1 = \text{step}hr$ and $hr_2 = \text{refl}$. Then, $A = C$, so $C \to_\beta B$. Recall forall x where x is a term, x is a normal for if $\neg\exists y, x \to_\beta y$. Hence, $B = C$ by contradiction.
- Refl, step case: Suppose there exists hr such that $hr_2 = \text{step}hr$ and $hr_1 = \text{refl}$. Then, $A = B$, so $B \to_\beta C$. Hence, $B = C$ by contradiction.
- Step, step case: Suppose there exists hr_A and hr_B such that $hr_1 = \text{step}hr_A$ and $hr_2 = \text{step}hr_B$. Then, by the inductive hypothesis, $B = C$.

☑ [Formalized: `Dcc.GenericExpr.norm_unique`] □

3.4 Decidability of $=_\beta$

We also demonstrate soundness and completeness of a β-reduction procedure (`reduce RedAff A` forall terms A) with respect to the step relation.

Theorem 6. *If A and B are terms such that $hr : A \to_\beta B$, then there exists C and $hr' : A \to_\beta C$ such that `step? A = some ⟨C, hr'⟩`.*

Proof. The proof proceeds by splitting on `step? A` with respect to the conclusion. The proof has been omitted for brevity. ☑ [Formalized: `Dcc.GenericExpr.semi_complete`] □

Theorem 7. *If A and B are terms such that $hr : A \to_\beta^* B$ and B is a normal form, then `reduce RedAff A = ⟨B, hr⟩`.*

Proof. Suppose A and B are terms such that $hr : A \to_\beta^* B$ and B is a normal form. The proof proceeds by splitting on the definition of `reduce`.

- `step? A = none` case: Suppose `step? A = none`. Then, `reduce A = ⟨A, refl⟩`. Also, there does not exist B such that $A \to_\beta B$. Hence, `hr = refl`.
- Case `step? A = ⟨C, hr'⟩`: Suppose `step? A = ⟨C, hr'⟩`. By global confluence 4, there exists D such that $B \to_\beta^* D$ and $hrt : C \to_\beta^* D$. Proceed by case analysis on hrt.
  - Refl case: By the hypothesis that C is β-normal, $C = D$. And by proof irrelevance, $hr = hr'$.
  - Step case: By 3.1 and `hr`, size(C) < size(A). So, by well-founded induction with respect to size, `reduce C = ⟨D, hrt⟩`. Hence, `step? A = reduce C` and $B \to_\beta^*$ (reduce$C$). So, since B is a normal form, $B =$ reduce$C$. Therefore, $C = B$, so `reduce RedAff A = ⟨B, hr⟩`.

☑ [Formalized: `Dcc.GenericExpr.reduce_complete`] □

Theorem 8. *Forall terms A and B, $hr : A \to_\beta^* B$ or $\neg A \to_\beta^* B$.*

Proof. By completeness and soundness of `reduce RedAff`. ☑ [Formalized: `Dcc.GenericExpr.reduce_complete`] □

4 η-reduction in the categorical combinators is extensional

Computable function extensionality is the natural consequence of η-reduction in DCC: there are no open terms, η-normal forms are mutually exclusive in behavior, η-normal forms correspond to operations in the cartesian closed category and are as such "well-behaved", and all η preimages are already β-normal due to strong normalization as a consequence of our affine reduction rules, judgments are canonical (no catch-all conversion), and all well-typed functions are η-reducible to normal forms (even if η-reduction is a no-op). There is exactly one η-normal form which demonstrates a particular behavior on any argument.

Our η-reduction procedure reduces functions to their canonical forms by observing their behavior on opaque "holes" whose shape matches the domain of the function given by term points. All function terms derive their domain from points. Type-checking is not required to calculate function domains. Our η-normal canonical forms are as such:

- Constant functions ‘(B, c) which do not contain the argument x after β-reduction
- The identity $Id(A)$, which gives x after β-reduction
- Couple projections, where $x[\![B]\!](y, z)$ β-reduces to x or y where $x \neq y$
- Or in case x accepts a boxed couple, $x\&([\![B]\!](y, z))$ reduces to x or y
- Evaluation, where $[\![B]\!](y, z)$ β-reduces to fxy
- Partial application, where xy (in case x is unary) or xyz (in case x is 2-arity) β-reduces to $x'y$ or $x'yz$, respectively
- The higher-order identity, where $Id(\Pi(B, C)\ \text{env})xy$ β-reduces to an application xy
- Unary quotation, where xy β-reduces to c and c does not contain y
- 2-arity quotation, where xyz β-reduces to c and c contains neither x nor y
- Boxed quotation, where x accepts boxed values and $x\&(y)\&(z)$ contains neither $\&(y)$ nor $\&(z)$

These canonical forms are mutually exclusive, and are complete with respect to all well-typed functions. For example, composition is handled by the partial application rule:

$$(Id(A) \circ' Id(A))x \to^!_\beta x \therefore Id(A) \circ' Id(A) \to_\eta Id(A) \tag{7}$$

No-op η-reductions are similarly handled by the partial application case, which is mutually exclusive in behavior with all other canonical forms. At a high level, since judgments are canonical due to avoiding catch-all conversion, and since we have proven all well-typed functions are η-reducible, and since all combinators have arity 2 or less, and since there are no open terms in combinator calculus, the η-normal forms of a given β-normal function are solely dictated by their behavior. Hence, function extensionality holds, and it is computable, not axiomatic.

Our approach is fully syntactic, although it bears some resemblance to normalization by evaluation (NbE) techniques.

4.1 Term points dictate function domains

We believe this approach is only possible due to our design pairing the categorical combinators with affine logic. In the SK calculus, for example, this approach is not possible in the SK calculus, since type arguments are given by application, not points. Function domains are encoded as a telescope context. The constituent terms are β-normal values. For example, $\Lambda(A, B, x)$ has domain $[A', B']$, where $B : A \to \text{Sort } v$ and $A \to^!_\beta A'$ and $B \to^!_\beta B'$.

Function domains are inferred as such .

EQL
$$\frac{\mathrm{dom}(B) = A}{\mathrm{dom}(\mathrm{Eql}(B)) :=!(A)}$$

UNBOX
$$\frac{}{\mathrm{dom}(*(A)) :=!(A)}$$

EQL1
$$\frac{Bx \to^!_\beta A}{\mathrm{dom}(\mathrm{Eql}(B))\&(x) :=!(A)}$$

REFL
$$\frac{\mathrm{dom}(B) = A}{\mathrm{dom}(\mathrm{refl}(B)) :=!(A)}$$

REFL1
$$\frac{Bx \to^!_\beta A}{\mathrm{dom}(\mathrm{refl}(B)\&(x)) :=!(A)}$$

UNMAP
$$\frac{\mathrm{dom}(x) = A}{\mathrm{dom}(*\langle x\rangle) :=!(A)}$$

FMAP
$$\frac{\mathrm{dom}(x) = A}{\mathrm{dom}(\$\langle x\rangle) :=!(A)}$$

MAP
$$\frac{\mathrm{dom}(x) = A}{\mathrm{dom}(!\langle x\rangle) :=!(A)}$$

ID
$$\frac{}{\mathrm{dom}(\mathrm{Id}(A)) := A}$$

FST
$$\frac{}{\mathrm{dom}(\mathrm{Fst}(B)) := \mathrm{Coup}(B)}$$

SND
$$\frac{}{\mathrm{dom}(\mathrm{Snd}(B)) := \mathrm{Coup}(B)}$$

CURR
$$\frac{}{\mathrm{dom}(\Lambda(A, B, x)) := A}$$

CURR1
$$\frac{Bx \to^!_\beta B'}{\mathrm{dom}(\Lambda(A, B, x)y) := B'}$$

QUOT
$$\frac{}{\mathrm{dom}(`(A, x)) := A}$$

CURR'
$$\frac{}{\mathrm{dom}(!\Lambda(A, B, x)) :=!(A)}$$

CURR'1
$$\frac{Bx \to^!_\beta B'}{\mathrm{dom}(!\Lambda(A, B, x)y) :=!(B')}$$

PAIR
$$\frac{\mathrm{dom}(x) = A}{\mathrm{dom}([\![B]\!]\langle x, y\rangle) := A}$$

PI
$$\frac{\mathrm{dom}(B) = A}{\mathrm{dom}(\Pi(B, C)) := A}$$

EVAL
$$\frac{}{\mathrm{dom}(\mathrm{Eval}(A, B, x)) := A}$$

EVAL1
$$\frac{By \to^!_\beta B'}{\mathrm{dom}(\mathrm{Eval}(A, B, x)y) := B'}$$

COMP'
$$\frac{\mathrm{dom}(g) = A}{\mathrm{dom}(f \circ' g) := A}$$

COMP
$$\frac{\mathrm{dom}(g) =!(A)}{\mathrm{dom}(f \circ g) :=!(A)}$$

COMP2
$$\frac{\mathrm{dom}(g) =!(A)}{\mathrm{dom}(f \circ \circ g) :=!(A)}$$

Figure 13: Where $\to_\eta$ represents η-reduction and $\to^!_\beta$ represents β-normal reduction. dom syntactically retrieves the β-reduced domain of a combinator given the environment. All domains are β-normal. Σ is omitted, since it has no η-reduction aside from reducing its points. We also model domains as a relation DomainxA with completeness and soundness with respect to dom(x). ☑ [Formalized: `Dcc.NormExpr.Domain`]

Theorem 9. *Forall x which is a term in β-normal form, if* `h : Domain x A`*, then* `domain? x = some ⟨A, h⟩`*.*

Proof. Omitted. Proceed by induction on h. ☑ [Formalized: `Dcc.NormExpr.Domain.complete`] □

Theorem 10. *Forall x which is a term in β-normal form, if* $\mathtt{h}_1$ `: Domain x A` *and* $\mathtt{h}_1$ `: Domain x B`*, then $A = B$.*

Proof. Suppose $\mathtt{h}_1$ `: Domain x A` and $\mathtt{h}_1$ `: Domain x B`. Then, `domain? x = some ⟨A,` $\mathtt{h}_1$`⟩` and `domain? x = some ⟨B,` $\mathtt{h}_2$`⟩`. Hence, $A = B$. This is due to completeness 4.1. □

Domains for 2-ary functions are calculated by ☑ [Formalized: `reducing` Bx], where x is a hole.

$$\textsc{Quot}\ \frac{}{\mathrm{dom}_2(\text{`}(A, \text{`}(B, x))) \coloneqq (A, B)} \qquad \textsc{Id}\ \frac{Bx \to^!_\beta A}{\mathrm{dom}_2(\mathrm{Id}(\Pi(B, C)x)) \coloneqq (\Pi(B, C)x, A)} \qquad \textsc{Comp'}\ \frac{\mathrm{dom}(g) = A, \mathrm{dom}_2(f) = (B, C)}{\mathrm{dom}_2(f \circ' g) \coloneqq (A, C \circ' f)}$$

$$\textsc{Eql}\ \frac{\mathrm{dom}(B) = A}{\mathrm{dom}_2(\mathrm{Eql}(B)) \coloneqq (A, B)} \qquad \textsc{Refl}\ \frac{\mathrm{dom}(B) = A}{\mathrm{dom}_2(\mathrm{refl}(B)) \coloneqq (A, B)} \qquad \textsc{Curr}\ \frac{\mathrm{dom}(x) = \mathrm{Coup}(B)}{\mathrm{dom}_2(\Lambda(A, B, x)) \coloneqq (A, B)}$$

$$\textsc{Curr'}\ \frac{\mathrm{dom}(x) = !(\mathrm{Coup}(B))}{\mathrm{dom}_2(!\Lambda(A, B, x)) \coloneqq (!(A), !\langle B\rangle)}$$

Figure 14: Arity 2 domains. The second projection of $\mathrm{dom}(2)x$ is a function of the first term argument. Double quotation is η-reducible and considered to have arity 2 when double quoted. Similarly, when Id accepts a function, it has arity 2. ☑ [Formalized: `Dcc.NormExpr.Domain`]

Theorem 11. *Forall x which is a term in β-normal form, if* `h : Domain₂ x A B`*, then* `domain₂? x = some ⌈A, B, h⌉`*.*

Proof. Omitted. Proceed by induction on h. ☑ [Formalized: `Dcc.NormExpr.Domain.complete`] □

Theorem 12. *Forall x which is a term in β-normal form, if* `h₁ : Domain₂ x A B` *and* `h₁ : Domain₂ x C D`*, then $A = C$ and $B = D$.*

Proof. Suppose `h₁ : Domain₂ x A B` and `h₁ : Domain₂ x C D`. Then, `domain₂? x = some ⟨A, B, h₁⟩` and `domain₂? x = some ⟨C, D, h₂⟩`. Hence, $A = C$ and $B = D$. This is due to completeness 4.1. □

4.2 Term points generate opaque holes

Our approach relies on the assumption that extensional "test values" are not contained in the function being reduced. Hence, we extend terms to contain "holes" as constant values. Holes are identified by a level, so as x may be distinguished from y when normalizing a couple projection. Terminal holes are combined via coupling and boxing to match the domain of the given function. Holes are generated as such:

$$\eta\textsc{-unary functions}\ \frac{\mathrm{arity}(x) = 1, \mathrm{dom}(x) = A}{\mathrm{F}_\eta\ x\ [A]} \qquad \eta\textsc{-binary functions}\ \frac{\mathrm{arity}(x) = 2, \mathrm{dom}_2(x) = (A, B)}{\mathrm{F}_\eta\ x\ [A, B]} \qquad \textsc{Affine bang holes}\ \frac{\mathrm{Hole}\ A(\mathrm{level} + 1)x, \mathbb{N}}{\mathrm{Hole}\ !(A)\mathrm{level}\&(x)}$$

$$\textsc{Constant holes}\ \frac{(\neg\exists B, A = \mathrm{Coup}(B)) \wedge (\neg\exists B, A = !(B)), \mathbb{N}}{\mathrm{Hole}\ A\ \mathrm{level}(\mathrm{const}(\mathrm{holelevel}))}$$

$$\textsc{Couple holes}\ \frac{\mathrm{dom}(B) = A, \mathrm{Hole}\ A(\mathrm{level} + 2)x, Bx \to^!_\beta B', \mathrm{Hole}\ B'(\mathrm{level} + 1)y, \mathbb{N}}{\mathrm{Hole}\ \mathrm{Coup}(B)\ \mathrm{level}[\![B]\!](x, y)}$$

Figure 15: Where $\mathrm{F}_\eta\ f$ doms represents a view of f as an η-reducible function with domain "doms". ☑ [Formalized: `Dcc.NormExpr.Hole`]

4.3 η-Canonicalization

Canonical η-normal values are mutually exclusive and deterministic. This fact is demonstrated by ☑ [Formalized: `completeness and soundness`] of our canonicalization implementation (including domain calculation and hole generation) with respect to our relational rules. We have omitted the proofs for brevity, as they are mostly repetitive case analysis and splitting on the respective definitions. Repeated η-reduction is a no-op.

$$\textsc{Id}\quad \frac{\mathrm{F}_\eta\ x\ [A], (\neg\exists BCy, x = \Pi(B,C)y), \mathrm{Hole}\ Ay, xy \to^!_\beta y}{x \to_\eta \mathrm{Id}(A)}$$

$$\textsc{Fst}\quad \frac{\mathrm{dom}(B) = A, \mathrm{F}_\eta\ x\ [[\mathrm{Coup}(B)], \mathrm{Hole}\ \mathrm{Coup}(B)([\![B]\!](y,z)), x[\![By]\!](y,z) \to^!_\beta y}{x \to_\eta \mathrm{Fst}(B)}$$

$$\textsc{Snd}\quad \frac{\mathrm{dom}(B)A, \mathrm{F}_\eta\ x\ [\mathrm{Coup}(B)], \mathrm{Hole}\ \mathrm{Coup}(B)([\![B]\!](y,z)), x[\![By]\!](y,z) \to^!_\beta z}{x \to_\eta \mathrm{Snd}(B)}$$

$$\textsc{Fst'}\quad \frac{\mathrm{dom}(B)A, \mathrm{F}_\eta\ x\ [!(\mathrm{Coup}(B))], \mathrm{Hole}\ !(\mathrm{Coup}(B))(\&([\![B]\!](y,z))), x\&([\![By]\!](y,z)) \to^!_\beta y}{x \to_\eta *\langle \mathrm{Fst}(B)\rangle}$$

$$\textsc{Snd'}\quad \frac{\mathrm{dom}(B)A, \mathrm{F}_\eta\ x\ [!(\mathrm{Coup}(B))], \mathrm{Hole}\ !(\mathrm{Coup}(B))(\&([\![B]\!](y,z))), x\&([\![By]\!](y,z)) \to^!_\beta z}{x \to_\eta *\langle \mathrm{Snd}(B)\rangle}$$

$$\textsc{Par}\quad \frac{\mathrm{dom}(A)y, ((\neg\exists A', A = \mathrm{Coup}(A'))(\neg\exists A', A = !(A'))), \mathrm{F}_\eta\ x\ [A], \mathrm{Hole}\ Ay, xy \to^!_\beta x'y, \mathrm{F}_\eta\ x'\ [A]}{x \to_\eta x'}$$

$$\textsc{Eval}\quad \frac{\mathrm{dom}(B)A, \mathrm{F}_\eta\ x\ [\mathrm{Coup}(B)], \mathrm{Hole}\ \mathrm{Coup}(B)[\![B]\!](y,z), x[\![B]\!](y,z) \to^!_\beta x'yz, \mathrm{F}_\eta\ x'\ [A,B]}{x \to_\eta \mathrm{Eval}(A,B,x')}$$

$$\textsc{Quot}\quad \frac{\mathrm{F}_\eta\ x\ [A], \mathrm{Hole}\ Ay, xy \to^!_\beta e', \neg\exists l, \mathrm{const}(\mathrm{hole} l) \in e'}{x \to_\eta \text{`}(A, e')}$$

$$\textsc{Id}_2\quad \frac{\mathrm{F}_\eta\ x\ [\Pi(f_A, f_B)\mathrm{env}, \text{`}(\Pi(f_A, f_B)\mathrm{env}, B)], \mathrm{Hole}\ (\Pi(f_A, f_B)\mathrm{env})y, \mathrm{Hole}\ Bz, xyz \to^!_\beta yz}{x \to_\eta \mathrm{Id}(\Pi(f_A, f_B)\mathrm{env})}$$

$$\textsc{Quot}_2\quad \frac{\mathrm{F}_\eta\ x\ [A, \text{`}(A,B)], \mathrm{Hole}\ Ay, \mathrm{Hole}\ Bz, xyz \to^!_\beta e', (\neg\exists l, \mathrm{const}(\mathrm{hole} l) \in e')}{x \to_\eta \text{`}(A, \text{`}(B, e'))}$$

$$\textsc{Bang Quot}\quad \frac{\mathrm{F}_\eta\ x\ [!(A), !\langle B\rangle], \mathrm{Hole}\ !(A)y, \langle B\rangle y \to^!_\beta B', \mathrm{Hole}\ B'z, xyz \to^!_\beta e', (\neg\exists l, \mathrm{const}(\mathrm{hole} c) \in e')}{x \to_\eta !\Lambda(A, B, \text{`}(!(\mathrm{Coup}(B)), e'))}$$

Figure 16: Where $\to_\eta$ represents function canonicalization. Hole-application observes function behavior. Functions are accordingly mapped to canonical operations which correspond to operations in the cartesian closed category. Holes are injected into the AST with a const constructor. ☑ [Formalized: `Dcc.NormExpr.EtaFn.Canonicalize`]

The preimages under η-reduction `x : @GenericExpr A` are mapped to identical terms in `@GenericExpr (@EtaConst A)`. `EtaConst` is inhabited by constants `EtaConst.const : A → (@EtaConst A)`, and `EtaHole` with constructors for sort holes and opaque holes: `hole n`, where n is a unique level, or `sort n`. The image under this mapping contains no holes, since existing constants are mapped to `@GenericExpr.const (@EtaConst.const x)`.

Theorem 13. *Forall terms `x : @GenericExpr A`, if `x` is mapped to `x' : @GenericExpr (@EtaConst A)`, then x' contains no holes.*

Proof. Suppose `x : @GenericExpr A`. By induction on x, x is either a DCC term, or a constant value:

- Concrete term case: by the inductive hypothesis, all of the term's points do not contain holes, and a conrete terms are not holes
- Constant case: if there exists some $c : A$ such that `x = const c`, then `x' = const (EtaConst.const c)`. Since `EtaConst.const` is not a hole, x' does not contain holes.

☑ [Formalized: `Dcc.GenericExpr.RedAff.tail_no_hole`] □

Theorem 14. *Forall x which is a term in β-normal form, if x does not contain any holes, and if `Domain x A`, then A does not contain any holes.*

Proof. Suppose x is a term in β-normal form which is the preimage under η-reduction, and that `h : Domain x A`. Proceed by induction on h. Since all domains are derived from term points, and since η-reducible terms do not contain holes, then A does not contain holes. ☑ [Formalized: `Dcc.NormExpr.Domain.no_holes`] □

Theorem 15. *Forall x which is a term in β-normal form, and if `Domain₂ x A B`, then neither A nor B contain holes.*

Proof. Suppose x is a term in β-normal form, and that `h : Domain₂ x A B`. Proceed by induction on h. Since all domains are derived from term points, and since η-reducible terms do not contain holes, then neither A nor B contain holes. ☑ [Formalized: `Dcc.NormExpr.Domain2.no_holes`] □

Theorem 16. *Forall x which is a term in β-normal form, where `h : Canonicalize x x'`, then `canonicalize? x = some ⟪x, h⟫`.*

Proof. Omitted. Proceed by induction on h and case splitting on the conclusion with respect to the definition of `canonicalize?`. ☑ [Formalized: `Dcc.NormExpr.EtaFn.Canonicalize.complete`] □

Theorem 17. *Forall x which is a term in β-normal form, if `Canonicalize x x₁` and `Canonicalize x x₂`, then `x₁ = x₂`.*

Proof. By completeness 16: if `h₁ : Canonicalize x x₁` and `h₂ : Canonicalize x x₂`, then `canonicalize? x = some ⟨x₁, h₁⟩` and `canonicalize? x = some ⟨x₂, h₂⟩`. Hence, `x₁ = x₂`. ☑ [Formalized: `Dcc.NormExpr.EtaFn.Canonicalize.unique`] □

4.4 η-reduction is well-behaved and canonical

We note that the set of possible η-canonical values is mutually exclusive in behavior, as in 17. That is, no two constructors share the same reduction when applied their respective holes. If $f \neq g$, and f and g are in β-η-normal form, then $\neg(fx =_\beta gx)$ for all x, where x is the appropriate hole determined by the domain of f and g as described in 4.1. By the contrapositive, if $(fx) =_\beta (gx)$, then $f = g$.

Theorem 18. *Forall `f₁` and `f₂` where `Canonicalize f₁ f₂`, `dom(f₁) = dom(f₂)`.*

Proof. Proceed by case analysis on the η-reduction relation. By construction, `dom(f₁) = dom(f₂)`. □

Theorem 19. *Forall unary functions `f₁` and `f₂` where `Canonicalize f₁ f₂` and Hole$f_1 x_1$ and Hole$f_2 x_2$, then $x_1 = x_2$.*

Proof. Suppose `f₁` and `f₂` are unary η-reducible functions identified with domains A and B, and are in η-normal form. Suppose `h₁ : Hole A x₁` and `h₂ : Hole B x₂`. Recall that by 18, $A = B$. Hence, `Hole A x₁ ∧ Hole A x₂`. By completeness of the hole-generation procedure with respect to the hole-generation relation, `hole A = ⟨x₁, h₁⟩` and `hole A = ⟨x₂, h₂⟩`. Hence, `x₁ = x₂`. ☑ [Formalized: `Dcc.NormExpr.Hole.complete`] □

4.5 η-reduction induces computable function extensionality

This proof is natural by the construction of η-normal forms, which are observably mutually exclusive in behavior. $f \to_\eta g$ only if g observably mirrors f applied its respectives holes.

Theorem 20. *Forall unary η-reducible terms f_1 and f_2 with domain A where `h₁ : Canonicalize f₁ f₂`, and where `Hole A x₁` and `Hole A x₂`, then $f_1 x_1 =_\beta f_2 x_2$.*

Proof. Suppose f_1 and f_2 are unary η-reducible terms identified by domain A and where `h₁ : Canonicalize f₁ f₂`. Also, suppose `Hole A x₁` and `Hole A x₂`. Then, by 19, $x_1 = x_2$. Proceed by case analysis on `h₁` 16:

- Id case: Suppose h is given by Id. Then, $f_1 x_1 =_\beta x_1$ and $f_2 = \mathrm{Id}(A)$. So, $f_2 x_1 =_\beta x_1$. Hence $f_1 x_1 =_\beta f_2 x_2$.
- Fst case: Suppose h is given by Fst. Then, there exists B such that $A = \mathrm{Coup}(A)$ and x_{1a} and x_{1b} such that $x_1 = [\![B]\!](x_{1a}, x_{1b})$. Then, $f_1[\![B]\!](x_{1a}, x_{1b}) =_\beta x_{1a}$ and $f_2 = \mathrm{Fst}(B)$. So, $f_2[\![B]\!](x_{1a}, x_{1b}) =_\beta x_{1a}$. Hence $f_1 x_1 =_\beta f_2 x_2$.
- Snd case: Suppose h is given by Snd. Then, there exists B such that $A = \mathrm{Coup}(A)$ and x_{1a} and x_{1b} such that $x_1 = [\![B]\!](x_{1a}, x_{1b})$. Then, $f_1[\![B]\!](x_{1a}, x_{1b}) =_\beta x_{1b}$ and $f_2 = \mathrm{Snd}(B)$. So, $f_2[\![B]\!](x_{1a}, x_{1b}) =_\beta x_{1b}$. Hence $f_1 x_1 =_\beta f_2 x_2$.
- Fst' case: Suppose h is given by Fst'. Then, there exists B such that $A = !(\mathrm{Coup}(A))$ and x_{1a} and x_{1b} such that $x_1 = \&([\![B]\!](x_{1a}, x_{1b}))$. Then, $f_1 \&([\![B]\!](x_{1a}, x_{1b})) =_\beta x_{1a}$ and $f_2 = *\langle \mathrm{Fst}(B) \rangle$. So, $f_2 \&([\![B]\!](x_{1a}, x_{1b}) =_\beta x_{1a})$. Hence $f_1 x_1 =_\beta f_2 x_2$.
- Snd' case: Suppose h is given by Snd'. Then, there exists B such that $A = !(\mathrm{Coup}(A))$ and x_{1a} and x_{1b} such that $x_1 = \&([\![B]\!](x_{1a}, x_{1b}))$. Then, $f_1 \&([\![B]\!](x_{1a}, x_{1b})) =_\beta x_{1b}$ and $f_2 = *\langle \mathrm{Snd}(B) \rangle$. So, $f_2 \&([\![B]\!](x_{1a}, x_{1b}) =_\beta x_{1b})$. Hence $f_1 x_1 =_\beta f_2 x_2$.
- Quot case: Suppose h is given by Snd'. Then, there exists c such that $f_1 x_1 =_\beta c$, where $x_1 \notin c$. Hence, $f_2 = `(A, c)$ and $f_2 x_1 = c$. Hence, $f_1 x_1 =_\beta f_2 x_2$.

□

Theorem 21. *Forall f, f_1, and f_2, where f, f_1, and f_2 are in normal form, and if `h₁ : Canonicalize f f₁ h₂ : Canonicalize f₁ f₂`, then $f_1 = f_2$.*

Proof. Recall that η-normal forms are mutually exclusive in reduction with respect to their corresponding holes 16. If $f \neq_\beta f$, then $f_1 x_1 \neq_\beta f_2 x_1$. However, by 20, $f_1 x_1 =_\beta f_2 x_1$. Therefore, by contradiction, $f_1 = f_2$. ☑ [Formalized: `Dcc.NormExpr.EtaFn.Canonicalize.is_norm_fn`] □

Theorem 22. *Suppose f and g are unary functions where f and g are in η-normal form by canonicalization. Then, if $fx =_\beta gx$, where x is the appropriate hole, then $f = g$.*

Proof. Suppose f and g are unary functions where f and g are in η-normal form. Recall by 20, if `Canonicalize f₁ f₂`, then $fx_1 =_\beta gx_1$, where x_1 is the appropriate hole identified by the domain of f and g 18. By the contrapositive, if $fx_1 \neq_\beta gx_1$, then $f \neq g$. Suppose $fx_1 \neq_\beta gx_1$, then $f \neq g$. However, since `Canonicalize f₁ f₂`, $fx_1 =_\beta gx_1$. Hence $f = g$ by contradiction. ☑ [Formalized: `Dcc.NormExpr.EtaFn.Canonicalize.ext₁`] □

Theorem 23. *Suppose f and g are binary functions where f and g are in η-normal form by canonicalization. Then, if $fx_1x_2 =_\beta gx_1x_2$, where x_1 and x_2 are the appropriate holes.*

Proof. Suppose f and g are binary functions where f and g are in η-normal form. Suppose `h : Canonicalize f₁ f₂` and that x_1 and x_2 are holes generated by the domain of f and g. Similarly to in 20, case analysis on h reveals that $fx =_\beta gx$. By the contrapositive, if $fx \neq_\beta gx$, then $f \neq g$. Suppose $fx_1 \neq_\beta gx_1$, then $f \neq g$. However, since `Canonicalize f₁ f₂`, $fx_1 =_\beta gx_1$. Hence $f = g$ by contradiction. ☑ [Formalized: `Dcc.NormExpr.EtaFn.Canonicalize.ext₁`] □

5 Decidable type-checking in extensional DTT

In DCC, there is exactly one valid judgment for a given term. Catch-all conversion is not permitted. Types are β-η reduced to normal forms eagerly in each inference rule. This is possible due to inheriting strong normalization as a consequence of the choice of affine logic. This fact has sweeping ramifications. In particular, the extensional equality reflection rule is possible.

5.1 Propositional equality reflection is a natural consequence of affine DTT

Recall the extensional equality reflection rule:

$$\frac{h : x = y}{x \equiv y} \quad \text{Eq. Reflect} \tag{8}$$

Note that our inference rules do not permit general conversion 9. Instead, definitional equality is constrained to inference of application:

$$\frac{f : (x : A_1) \to Bx, x : A_2, A_1 \equiv A_2, (Bx) \to^!_\beta C, C \to^!_\eta D}{fx : D} \quad \text{App} \tag{9}$$

Also recall the inference rule for ref:

$$\text{REFL}\ \frac{B : A \to \text{Sort } u}{\text{refl}(B) : (x :!(A)) \to (y : (!\langle B \rangle x)) \to (\text{Eql}(B)xx)} \quad \therefore \quad \text{REFL APP}\ \frac{B : A \to \text{Sort } u, x :!(A), y :!(Bx)}{\text{refl}(B)\&(x)\&(y) :\to ((\text{Eql}(B)\&(x)\&(y)) =_\beta x = x)}$$

Intuitively, since refl is the only inference rule that generates a proof of equality, and since judgments are canonical in that exactly one judgment is valid for a given term, and since $x = y$ iff , then $x = y \iff x \equiv y$, where definitional equality is as defined in 9. Furthermore, $h : x = y$ induces a syntactic equality as a metatheorem in Lean 4, since x and y are eagerly reduced to β-η form forall x and y. That is, propositional equality is equivalent to definitional and meta-level syntactic equality. A propositional equality in DCC induces a definitional equality on the AST itself.

This property appears superficially restrictive. However, conversion is allowed in the inference rule for application, but not in general as a catch-all rule 9. Hence, this type-checks:

```
  #eval (DCC Id(Id(⊤) ⋆ = ⋆) (refl(Id(Sort 0)) ⊤ ⋆)).infer'
/-
Messages here:
⋆ = ⋆
-/
```

where `Id(Id(⊤) ⋆ = ⋆)` expects a proof that `Id(⊤) ⋆ = ⋆`. That is, reflexivity is a valid proof of `Id(⊤) ⋆ = ⋆` in argument position, but is ultimately reduced to β-η normal form, inducing the meta-level equality:

```
Expr.tintro = Expr.tintro
```

5.2 A type-checker for DCC

We outline our implementation of a type-checker which has soundness and completeness with respect to the judgment relation at a high-level. The full implementation is available in the appendix A.

Definitional equality is defined as such (as in the app inference rule):

```
inductive Reduces.{u} {τ_c : Type u} : GenericExpr τ_c → GenericExpr τ_c → Type u
  | mk : RedAffNorm A B
    → B.raw.RedEta C
    → Reduces A C

inductive DefEq.{u} {τ_c : Type u} [DecidableEq τ_c] [BEq τ_c] : GenericExpr τ_c →
↪  GenericExpr τ_c → Prop
  | pi      : Reduces (DCC? (f_α₁ env₁)) f_α'
    → Reduces (DCC? (f_α₂ env₂)) f_α'
    → Reduces (DCC? (f_β₁ env₁)) f_β'
```

```
    → Reduces (DCC? (f_β₂ env₂)) f_β'
    → DefEq (DCC? (Π(f_α₁, f_β₁) env₁)) (DCC? (Π(f_α₂, f_β₂) env₂))
  | bang    : DefEq τ₁ τ₂ → DefEq (DCC? !(τ₁)) (DCC? !(τ₂))
  | coup    : DefEq τ₁ τ₂ → DefEq (DCC? Coup(τ₁)) (DCC? Coup(τ₂))
  | pi_comp : Reduces (DCC? (Π(f_α₁, f_β₁) ∘' g₁)) τ'
    → Reduces (DCC? (Π(f_α₂, f_β₂) ∘' g₂)) τ'
    → DefEq (DCC? (Π(f_α₁, f_β₁) ∘' g₁)) (DCC? (Π(f_α₂, f_β₂) ∘' g₂))
  | tm : (IsEtaTm τ₁ ⊕ IsEtaTm τ₂)
    → Reduces τ₁ τ'
    → Reduces τ₂ τ'
    → DefEq τ₁ τ₂
```

☑ [Formalized: `Dcc.GenericExpr.DefEq`]

The judgment relation is defined as such (some constructors have been omitted for brevity):

```
@[dcc_judge] abbrev refl.type.{u} {τ_c : Type u} (f_α δ : GenericExpr τ_c) (o : Level) :
↪  GenericExpr τ_c :=
  (DCC? (Π(Id(Sort o), `(Sort o, Π(!<f_α>, Eql(f_α)))) !(δ)))

@[dcc_judge] abbrev id.type.{u} {τ_c : Type u} (α : GenericExpr τ_c) (m : Level) :
↪  GenericExpr τ_c :=
  DCC? (α : Sort m) → α

syntax "(" gen_dcc ":" gen_dcc ")" "→" gen_dcc : gen_dcc

macro_rules
  | `(DCC[$ty:gen_dcc_loc] (: Sort $m) → $β:gen_dcc) =>
    `(DCC[$ty:gen_dcc_loc] Λ(Sort $m, Id(Sort $m), `(Coup(Id(Sort $m)), $β)))

@[dcc_judge] abbrev curr.type.{u} {τ_c : Type u} (α β γ : GenericExpr τ_c) (m : Level) :
↪  GenericExpr τ_c :=
  DCC? (Π(Id(Sort m), `(Sort m, Π(β, Λ(α, β, γ)))) α) -- weaken α, keep x

@[dcc_judge] abbrev Eval.type.{u} {τ_c : Type u} (α β γ : GenericExpr τ_c) (m n : Level) :
↪  GenericExpr τ_c :=
  DCC? (Π(Id(Sort (max m n)), `(Sort (max m n), Eval(α, β, γ))) Coup(β))

inductive Pi.f_β.type.{u} {τ_c : Type u} :
  (GenericExpr τ_c) → (GenericExpr τ_c) → Level → Level → (GenericExpr τ_c) → Type u
  | canon  : Pi.f_β.type f_α δ o n
    (DCC? (Π(Id(Sort o), `(Sort o, Π(f_α, Λ(δ, f_α, `(Coup(f_α), Sort n))))) δ))
  | canon' : Pi.f_β.type (DCC? !<f_α>) (DCC? !(δ)) o n
    (DCC? (Π(Id(Sort o), `(Sort o, Π(!<f_α>, !Λ(δ, f_α, `(!(Coup(f_α)), Sort n))))) !(δ)))
  | semi   : Pi.f_β.type f_α δ o n (DCC? (Π(Id(Sort o), `(Sort o, Π(f_α, `(δ, `(f_α₂, Sort
  ↪  n))))) δ))
  | arrow  : Pi.f_β.type (DCC? Id(Sort m)) δ o n
    (DCC? (Π(Id(Sort o), `(Sort o, `(δ, (Π(Id(Sort m), `(Sort m, `(α, Sort n))) α)))) δ))

@[dcc_judge] abbrev Pi.type.{u} {τ_c : Type u} (α : GenericExpr τ_c) (o m n : Level) :
↪  GenericExpr τ_c :=
  DCC? (α : Sort o) → Sort (max m n)

inductive Judge.{u} {τ_c : Type u} [DecidableEq τ_c] [BEq τ_c] : GenericExpr τ_c →
↪  GenericExpr τ_c → Type u where
  | eq     : Judge x α
    → Judge y α
    → Judge (DCC? x = y) (DCC? Sort 0)
  | refl   : Judge f_α τ_f_α
    → Arrow δ (DCC? Sort m) o τ_f_α
    → Reduces (refl.type f_α δ o) τ
    → Judge (DCC? refl(f_α)) τ
  | id     : Judge α (DCC? Sort m)
    → Reduces α α'
    → Reduces (id.type α' m) τ
    → Judge (DCC? Id(α)) τ
  | curr   : Judge α (DCC? Sort m)
    → Judge x (DCC? (Π(f_α, γ) x_α))
    → Reduces (DCC? Coup(β)) τα
    → Reduces (DCC? (f_α x_α)) τα
    → Reduces (DCC? (γ x_α)) γ'
    → Reduces α α'
    → Reduces (curr.type α' β γ' m) τ
    → Judge (DCC? Λ(α, β, x)) τ
  | eval   : Judge x (DCC? (Π(f_α, f_β) env))
    → Reduces (DCC? (f_α env)) α
```

```
    → Reduces (DCC? (f_β env)) (DCC? (Π(β, γ)))
    → Judge α (DCC? Sort m)
    → Judge β (DCC? (Π(f_α₂, f_β₂) env₂))
    → Reduces (DCC? (f_α₂ env₂)) α
    → Reduces (DCC? (f_β₂ env₂)) (DCC? `(α, Sort n))
    → Reduces (Eval.type α β γ m n) τ
    → Judge (DCC? Eval(α, β, x)) τ
  | pi     : Judge f_α τ_f_α
    → Arrow δ (DCC? Sort m) o τ_f_α
    → Judge f_β τ_f_β
    → Reduces f_α f_α'
    → Pi.f_β.type f_α' δ o n τ_f_β
    → Reduces (Pi.type δ o m n) τ
    → Judge (DCC? Π(f_α, f_β)) τ
  | sort   : Judge (DCC? Sort m) (DCC? Sort m.succ)
  | top    : Judge (DCC? ⊤) (DCC? Sort 0)
  | bot    : Judge (DCC? ⊥) (DCC? Sort 0)
  | tintro : Judge (DCC? *) (DCC? ⊤)
  | bang   : Judge τ (DCC? Sort m) → Judge (DCC? !(τ)) (DCC? Sort m)
  | app    : Judge f (DCC? (Π(f_α, f_β) x_α))
    → Reduces (DCC? (f_α x_α)) α₁
    → Judge x α₂
    → DefEq α₁ α₂
    → Reduces x_α x_α'
    → Reduces (DCC? (f_β x_α' x)) τ
    → Judge (DCC? (f x)) τ
```

☑ [Formalized: `Dcc.GenericExpr.Judge`]

Note that convesrion is restricted to type-checking applications, and we do not assume general catch-all conversion. Also, judgments live in Type u, rather than *Prop*, enabling extensional equality elimination as a computable definition.

The type-checker is implemented as such (only the above cases are included for brevity).

```
def infer.{u} {τ_c : Type u} [hDecEq : DecidableEq τ_c] [BEq τ_c] (e : GenericExpr τ_c) :
↪  Except (TypeError e) ((t : GenericExpr τ_c) × Judge e t) :=
  match e with
  | DCC? ⊤ => .ok ⟨_, .top⟩
  | DCC? ⊥ => .ok ⟨_, .bot⟩
  | DCC? * => .ok ⟨_, .tintro⟩
  | DCC? Sort _m => .ok ⟨_, .sort⟩
  | DCC? (Id(α)) =>
    match infer α with
    | .ok ⟨DCC? Sort _m, ht⟩ => .ok ⟨_, .id ht (reduce_reduces rfl) (reduce_reduces rfl)⟩
    | .ok ⟨t, _⟩ => .error (.unexpected t α)
    | .error ε => .error (.sub α ε)
  | DCC? x = y =>
    match infer x with
    | .ok ⟨t_x, ht₁⟩ =>
      match infer y with
      | .ok ⟨t_y, ht₂⟩ =>
        if heq : t_x = t_y then
          .ok ⟨_, .eq ht₁ (heq ▸ ht₂)⟩
        else .error (.mismatch t_x t_y y)
      | .error ε => .error (.sub y ε)
    | .error ε => .error (.sub x ε)
  | DCC? refl(f_α) =>
    match infer f_α with
    | .ok ⟨τ, ht₁⟩ =>
      match τ.read_arrow with
      | .some ⟨_δ, DCC? Sort _m, _o, h⟩ =>
        .ok ⟨_, .refl ht₁ h (reduce_reduces rfl)⟩
      | _ => .error (.unexpected τ f_α)
    | .error ε => .error (.sub f_α ε)
  | DCC? Λ(α, β, x) =>
    match α.infer with
    | .ok ⟨DCC? Sort _m, ht⟩ =>
      match infer x with
      | .ok ⟨DCC? (Π(f_α, _γ) x_α), ht₂⟩ =>
        if heq : reduce_all (DCC? (f_α x_α)) = reduce_all (DCC? Coup(β)) then
          .ok ⟨_, .curr
            ht
            ht₂
            (reduce_reduces heq.symm)
            (reduce_reduces rfl)
```

```
          (reduce_reduces rfl)
          (reduce_reduces rfl)
          (reduce_reduces rfl)⟩
      else .error (.mismatch (reduce_all (DCC? Coup(β))) (reduce_all (DCC? (f_α x_α))) x)
    | .ok ⟨t, _⟩ => .error (.unexpected t x)
    | .error ε => .error (.sub x ε)
  | .ok ⟨t, _⟩ => .error (.unexpected t α)
  | .error ε => .error (.sub α ε)
| DCC? Eval(α, β, x) =>
  match infer x with
  | .ok ⟨DCC? (Π(f_α, f_β) env), ht₁⟩ =>
    if heq : (DCC? (f_α env)).reduce_all = α then
      match heq₂ : (DCC? (f_β env)).reduce_all with
      | DCC? Π(β₂, _γ) =>
        if heq₃ : β₂ = β then
          match infer α with
          | .ok ⟨DCC? Sort _m, ht₂⟩ =>
            match infer β with
            | .ok ⟨DCC? (Π(f_α₂, f_β₂) env₂), ht₃⟩ =>
              if heq₄ : (DCC? (f_α₂ env₂)).reduce_all = α then
                match heq₅ : (DCC? (f_β₂ env₂)).reduce_all with
                | DCC? `(α₃, Sort _n) =>
                  if heq₆ : α₃ = α then
                    .ok ⟨_, .eval ht₁ (reduce_reduces heq) (heq₃ ▸ (reduce_reduces heq₂))
                      ht₂ ht₃ (reduce_reduces heq₄) (reduce_reduces (heq₆ ▸ heq₅))
                      ↪  (reduce_reduces rfl)⟩
                  else .error (.mismatch α α₃ β)
                | other => .error (.unexpected other β)
              else .error (.mismatch α (DCC? (f_α₂ env₂)).reduce_all β)
            | .ok ⟨t, _⟩ => .error (.unexpected t β)
            | .error ε => .error (.sub β ε)
          | .ok ⟨t, _⟩ => .error (.unexpected t α)
          | .error ε => .error (.sub α ε)
        else .error (.mismatch β β₂ x)
      | other => .error (.unexpected other x)
    else .error (.mismatch α (DCC? (f_α env)).reduce_all x)
  | .ok ⟨t, _⟩ => .error (.unexpected t x)
  | .error ε => .error (.sub x ε)
| DCC? Π(f_α, f_β) =>
  match f_α.infer with
  | .ok ⟨τ_f_α, ht₁⟩ =>
    match τ_f_α.read_arrow with
    | .some ⟨δ, DCC? Sort _m, o, harr⟩ =>
      match infer f_β with
      | .ok ⟨τ_f_β, ht₂⟩ =>
        match τ_f_β.view_pi_f_β f_α.reduce_all δ o with
        | .some ⟨_n, h₂⟩ =>
          .ok ⟨_, .pi ht₁ harr ht₂ (reduce_reduces rfl) h₂ (reduce_reduces rfl)⟩
        | _ => .error (.mark 4 (.unexpected τ_f_β f_β))
      | .error ε => .error (.mark 3 (.sub f_β ε))
    | _ => .error (.mark 2 (.unexpected τ_f_α f_α))
  | .error ε => .error (.mark 1 (.sub f_α ε))
| DCC? (f x) =>
  match infer f with
  | .ok ⟨DCC? (Π(f_α, _f_β) x_α), ht₁⟩ =>
    match infer x with
    | .ok ⟨τ₂', ht₂⟩ =>
      match def_eq? (reduce_all (DCC? (f_α x_α))) τ₂' with
      | .ok ⟨h⟩ =>
        .ok ⟨_, .app ht₁ (reduce_reduces rfl) ht₂ h (reduce_reduces rfl) (reduce_reduces
        ↪  rfl)⟩
      | .error ε => .error (.def_eq ε)
    | .error ε => .error (.sub x ε)
  | .ok ⟨t, _⟩ => .error (.unexpected t f)
  | .error ε => .error (.sub f ε)
| .const c => .error (.unexpected (.const c) (.const c))
```

☑ [Formalized: `Dcc.GenericExpr.infer`]

Due to lacking general conversion, completeness of `infer` with respect to `Judge` is easy to prove by induction on the judgment relation and case splitting on the definition of `infer`.

Theorem 24. *Forall terms x and τ, if* `h : Judge e τ`*, then* `infer e = ok ⟨τ, h⟩`*.*

Proof. By induction on the judgment relation and case splitting on the definition of `infer`:

```
def Judge.complete (h : Judge (τ_c := τ_c) e τ) : e.infer = .ok ⟨τ, h⟩ := by
  induction h
  case eq =>
    dsimp only [infer]
    dcc_aesop!
  case refl =>
    expose_names
    dsimp only [infer]
    simp only [a_ih, a_1.read_complete, a_2.eq]
    grind
  case id =>
    expose_names
    dsimp only [infer]
    simp only [a_ih, a_1.eq, a_2.eq]
    grind
  case curr =>
    expose_names
    dsimp only [infer]
    simp only [a_ih, a_ih_1, a_3.complete, a_3.eq, a_5.eq, a_2.complete, a_2.eq, a_4.eq,
    ↪  a_6.eq]
    grind
  case eval =>
    expose_names
    dsimp only [infer]
    simp only [a_ih, a_ih_1, a_ih_2, a_1.eq, a_2.eq, a_5.eq, a_6.eq, a_7.eq]
    grind
  case pi =>
    expose_names
    dsimp only [infer]
    simp only [a_ih, a_ih_1, a_1.read_complete, a_3.eq, a_5.eq]
    cases a_3.complete
    cases a_5.complete
    simp only [a_4.complete]
  case tintro | top | bot | sort => dsimp only [infer]
  case app =>
    expose_names
    dsimp only [infer]
    simp only [a_ih, a_ih_1, a_1.eq, a_3.eq, a_4.eq]
    cases a_1.complete
    cases a_3.complete
    cases a_4.complete
    simp only [h.complete]
```

Cases omitted above are available on GitHub. ☑ [Formalized: `Dcc.GenericExpr.Judge.complete`] □

5.3 Judgments are Canonical

Theorem 25. *Forall terms x, τ_1, and τ_2, if $x : \tau_1$ and $x : \tau_2$, then $\tau_1 = \tau_2$.*

Proof. Suppose x, τ_1, and τ_2 are terms, such that h_1 is a proof that $x : \tau_1$ and h_2 is a proof that $x : \tau_2$. Then, by judgmental completeness 24, `infer x = ok ⟨τ₁, h₁⟩` and `infer x = ok ⟨τ₂, h₂⟩`. Hence, $\tau_1 = \tau_2$. □

6 Examples of extensionality in DCC

7 Comparison to observational type theory and cubical type theory

8 Conclusions & Future Work

A Supplementary Materials

- **Lean 4 formalization repository:** https://github.com/lexzaiello/dcc
- **Type-checker implementation:** https://github.com/lexzaiello/DCC/blob/6438e9e4960086082714e3498f59b7373

A.1 Dependent K Compilation

```
(((((!Λ(⊤, `(⊤, Sort 3), !Λ(Coup(`(⊤, Sort 3)), Π(Snd(`(⊤, Sort 3)), Λ(Coup(`(⊤, Sort 3)),
↪  Snd(`(⊤, Sort 3)), `(Coup(Snd(`(⊤, Sort 3))), Sort 2))), !Λ(Coup(Π(Snd(`(⊤, Sort 3)),
Λ(Coup(`(⊤, Sort 3)), Snd(`(⊤, Sort 3)), `(Coup(Snd(`(⊤, Sort 3))), Sort 2)))), (Snd(`(⊤,
↪  Sort 3)) ∘'
Fst(Π(Snd(`(⊤, Sort 3)), Λ(Coup(`(⊤, Sort 3)), Snd(`(⊤, Sort 3)), `(Coup(Snd(`(⊤, Sort 3))),
↪  Sort 2))))), Λ(!(Coup((Snd(`(⊤, Sort 3)) ∘' Fst(Π(Snd(`(⊤, Sort 3)), Λ(Coup(`(⊤, Sort
↪  3)), Snd(`(⊤, Sort 3)), `(Coup(Snd(`(⊤, Sort 3))), Sort 2)))))), (Eval(Coup((Snd(`(⊤,
↪  Sort 3)) ∘' Fst(Π(Snd(`(⊤, Sort 3)),
Λ(Coup(`(⊤, Sort 3)), Snd(`(⊤, Sort 3)), `(Coup(Snd(`(⊤, Sort 3))), Sort 2)))))), ((Snd(`(⊤,
↪  Sort 3)) ∘' Fst(Π(Snd(`(⊤, Sort 3)), Λ(Coup(`(⊤, Sort 3)), Snd(`(⊤, Sort 3)),
↪  `(Coup(Snd(`(⊤, Sort 3))), Sort 2))))) ∘'
Fst((Snd(`(⊤, Sort 3)) ∘' Fst(Π(Snd(`(⊤, Sort 3)), Λ(Coup(`(⊤, Sort 3)), Snd(`(⊤, Sort 3)),
↪  `(Coup(Snd(`(⊤, Sort 3))), Sort 2))))))), (Snd(Π(Snd(`(⊤, Sort 3)), Λ(Coup(`(⊤, Sort 3)),
↪  Snd(`(⊤, Sort 3)),
`(Coup(Snd(`(⊤, Sort 3))), Sort 2)))) ∘' Fst((Snd(`(⊤, Sort 3)) ∘' Fst(Π(Snd(`(⊤, Sort 3)),
↪  Λ(Coup(`(⊤, Sort 3)), Snd(`(⊤, Sort 3)), `(Coup(Snd(`(⊤, Sort 3))), Sort 2))))))) ∘∘
↪  ⟦Λ(Coup((Snd(`(⊤, Sort 3)) ∘'
Fst(Π(Snd(`(⊤, Sort 3)), Λ(Coup(`(⊤, Sort 3)), Snd(`(⊤, Sort 3)), `(Coup(Snd(`(⊤, Sort 3))),
↪  Sort 2))))), `(Coup((Snd(`(⊤, Sort 3)) ∘' Fst(Π(Snd(`(⊤, Sort 3)), Λ(Coup(`(⊤, Sort 3)),
↪  Snd(`(⊤, Sort 3)),
`(Coup(Snd(`(⊤, Sort 3))), Sort 2))))), Coup((Snd(`(⊤, Sort 3)) ∘' Fst(Π(Snd(`(⊤, Sort 3)),
↪  Λ(Coup(`(⊤, Sort 3)), Snd(`(⊤, Sort 3)), `(Coup(Snd(`(⊤, Sort 3))), Sort 2))))))),
↪  (((Snd(`(⊤, Sort 3)) ∘'
Fst(Π(Snd(`(⊤, Sort 3)), Λ(Coup(`(⊤, Sort 3)), Snd(`(⊤, Sort 3)), `(Coup(Snd(`(⊤, Sort 3))),
↪  Sort 2))))) ∘' Fst((Snd(`(⊤, Sort 3)) ∘' Fst(Π(Snd(`(⊤, Sort 3)), Λ(Coup(`(⊤, Sort 3)),
↪  Snd(`(⊤, Sort 3)),
`(Coup(Snd(`(⊤, Sort 3))), Sort 2))))))) ∘' Fst(`(Coup((Snd(`(⊤, Sort 3)) ∘' Fst(Π(Snd(`(⊤,
↪  Sort 3)), Λ(Coup(`(⊤, Sort 3)), Snd(`(⊤, Sort 3)), `(Coup(Snd(`(⊤, Sort 3))), Sort
↪  2))))))), Coup((Snd(`(⊤, Sort 3)) ∘'
Fst(Π(Snd(`(⊤, Sort 3)), Λ(Coup(`(⊤, Sort 3)), Snd(`(⊤, Sort 3)), `(Coup(Snd(`(⊤, Sort 3))),
↪  Sort 2)))))))))⟧<Id(Coup((Snd(`(⊤, Sort 3)) ∘' Fst(Π(Snd(`(⊤, Sort 3)), Λ(Coup(`(⊤, Sort
↪  3)), Snd(`(⊤, Sort 3)),
`(Coup(Snd(`(⊤, Sort 3))), Sort 2))))))), Λ(Coup((Snd(`(⊤, Sort 3)) ∘' Fst(Π(Snd(`(⊤, Sort
↪  3)), Λ(Coup(`(⊤, Sort 3)), Snd(`(⊤, Sort 3)), `(Coup(Snd(`(⊤, Sort 3))), Sort 2))))),
↪  `(Coup((Snd(`(⊤, Sort 3)) ∘'
Fst(Π(Snd(`(⊤, Sort 3)), Λ(Coup(`(⊤, Sort 3)), Snd(`(⊤, Sort 3)), `(Coup(Snd(`(⊤, Sort 3))),
↪  Sort 2))))), Coup((Snd(`(⊤, Sort 3)) ∘' Fst(Π(Snd(`(⊤, Sort 3)), Λ(Coup(`(⊤, Sort 3)),
↪  Snd(`(⊤, Sort 3)),
`(Coup(Snd(`(⊤, Sort 3))), Sort 2))))))), (Snd((Snd(`(⊤, Sort 3)) ∘' Fst(Π(Snd(`(⊤, Sort 3)),
↪  Λ(Coup(`(⊤, Sort 3)), Snd(`(⊤, Sort 3)), `(Coup(Snd(`(⊤, Sort 3))), Sort 2))))) ∘'
Fst(`(Coup((Snd(`(⊤, Sort 3)) ∘' Fst(Π(Snd(`(⊤, Sort 3)), Λ(Coup(`(⊤, Sort 3)), Snd(`(⊤, Sort
↪  3)),
`(Coup(Snd(`(⊤, Sort 3))), Sort 2))))), Coup((Snd(`(⊤, Sort 3)) ∘' Fst(Π(Snd(`(⊤, Sort 3)),
Λ(Coup(`(⊤, Sort 3)), Snd(`(⊤, Sort 3)), `(Coup(Snd(`(⊤, Sort 3))), Sort 2)))))))⟩),
((Snd((Snd(`(⊤, Sort 3)) ∘' Fst(Π(Snd(`(⊤, Sort 3)), Λ(Coup(`(⊤, Sort 3)), Snd(`(⊤, Sort 3)),
↪  `(Coup(Snd(`(⊤, Sort 3))), Sort 2))))) ∘' *(Coup((Snd(`(⊤, Sort 3)) ∘' Fst(Π(Snd(`(⊤,
↪  Sort 3)),
Λ(Coup(`(⊤, Sort 3)), Snd(`(⊤, Sort 3)), `(Coup(Snd(`(⊤, Sort 3))), Sort 2))))))) ∘'
↪  Fst((Eval(Coup((Snd(`(⊤, Sort 3)) ∘' Fst(Π(Snd(`(⊤, Sort 3)), Λ(Coup(`(⊤, Sort 3)),
↪  Snd(`(⊤, Sort 3)), `(Coup(Snd(`(⊤, Sort 3))), Sort 2))))),
((Snd(`(⊤, Sort 3)) ∘' Fst(Π(Snd(`(⊤, Sort 3)), Λ(Coup(`(⊤, Sort 3)), Snd(`(⊤, Sort 3)),
↪  `(Coup(Snd(`(⊤, Sort 3))), Sort 2))))) ∘' Fst((Snd(`(⊤, Sort 3)) ∘' Fst(Π(Snd(`(⊤, Sort
↪  3)),
Λ(Coup(`(⊤, Sort 3)), Snd(`(⊤, Sort 3)), `(Coup(Snd(`(⊤, Sort 3))), Sort 2))))))),
↪  (Snd(Π(Snd(`(⊤, Sort 3)), Λ(Coup(`(⊤, Sort 3)), Snd(`(⊤, Sort 3)), `(Coup(Snd(`(⊤, Sort
↪  3))), Sort 2)))) ∘' Fst((Snd(`(⊤, Sort 3)) ∘'
Fst(Π(Snd(`(⊤, Sort 3)), Λ(Coup(`(⊤, Sort 3)), Snd(`(⊤, Sort 3)), `(Coup(Snd(`(⊤, Sort 3))),
↪  Sort 2)))))))) ∘∘ ⟦Λ(Coup((Snd(`(⊤, Sort 3)) ∘' Fst(Π(Snd(`(⊤, Sort 3)), Λ(Coup(`(⊤, Sort
↪  3)), Snd(`(⊤, Sort 3)),
`(Coup(Snd(`(⊤, Sort 3))), Sort 2))))), `(Coup((Snd(`(⊤, Sort 3)) ∘' Fst(Π(Snd(`(⊤, Sort
↪  3)), Λ(Coup(`(⊤, Sort 3)), Snd(`(⊤, Sort 3)), `(Coup(Snd(`(⊤, Sort 3))), Sort 2))))),
↪  Coup((Snd(`(⊤, Sort 3)) ∘'
Fst(Π(Snd(`(⊤, Sort 3)), Λ(Coup(`(⊤, Sort 3)), Snd(`(⊤, Sort 3)), `(Coup(Snd(`(⊤, Sort 3))),
↪  Sort 2))))))), (((Snd(`(⊤, Sort 3)) ∘' Fst(Π(Snd(`(⊤, Sort 3)), Λ(Coup(`(⊤, Sort 3)),
↪  Snd(`(⊤, Sort 3)),
`(Coup(Snd(`(⊤, Sort 3))), Sort 2))))) ∘' Fst((Snd(`(⊤, Sort 3)) ∘' Fst(Π(Snd(`(⊤, Sort 3)),
↪  Λ(Coup(`(⊤, Sort 3)), Snd(`(⊤, Sort 3)), `(Coup(Snd(`(⊤, Sort 3))), Sort 2))))))) ∘'
↪  Fst(`(Coup((Snd(`(⊤, Sort 3)) ∘'
Fst(Π(Snd(`(⊤, Sort 3)), Λ(Coup(`(⊤, Sort 3)), Snd(`(⊤, Sort 3)), `(Coup(Snd(`(⊤, Sort 3))),
↪  Sort 2))))), Coup((Snd(`(⊤, Sort 3)) ∘' Fst(Π(Snd(`(⊤, Sort 3)), Λ(Coup(`(⊤, Sort 3)),
↪  Snd(`(⊤, Sort 3)),
```

```
`(Coup(Snd(`(⊤, Sort 3))), Sort 2))))))))))⟧⟨Id(Coup((Snd(`(⊤, Sort 3)) ∘' Fst(Π(Snd(`(⊤,
↪  Sort 3)), Λ(Coup(`(⊤, Sort 3)), Snd(`(⊤, Sort 3)), `(Coup(Snd(`(⊤, Sort 3))),
Sort 2))))))), Λ(Coup((Snd(`(⊤, Sort 3)) ∘' Fst(Π(Snd(`(⊤, Sort 3)), Λ(Coup(`(⊤, Sort 3)),
Snd(`(⊤, Sort 3)), `(Coup(Snd(`(⊤, Sort 3))), Sort 2))))))), `(Coup((Snd(`(⊤, Sort 3)) ∘'
Fst(Π(Snd(`(⊤, Sort 3)), Λ(Coup(`(⊤, Sort 3)), Snd(`(⊤, Sort 3)), `(Coup(Snd(`(⊤, Sort 3))),
Sort 2))))))), Coup((Snd(`(⊤, Sort 3)) ∘' Fst(Π(Snd(`(⊤, Sort 3)), Λ(Coup(`(⊤, Sort 3)),
Snd(`(⊤, Sort 3)), `(Coup(Snd(`(⊤, Sort 3))), Sort 2))))))), (Snd((Snd(`(⊤, Sort 3)) ∘'
Fst(Π(Snd(`(⊤, Sort 3)), Λ(Coup(`(⊤, Sort 3)), Snd(`(⊤, Sort 3)), `(Coup(Snd(`(⊤, Sort 3))),
Sort 2))))) ∘' Fst(`(Coup((Snd(`(⊤, Sort 3)) ∘' Fst(Π(Snd(`(⊤, Sort 3)), Λ(Coup(`(⊤, Sort
↪  3)),
Snd(`(⊤, Sort 3)), `(Coup(Snd(`(⊤, Sort 3))), Sort 2))))), Coup((Snd(`(⊤, Sort 3)) ∘'
Fst(Π(Snd(`(⊤, Sort 3)), Λ(Coup(`(⊤, Sort 3)), Snd(`(⊤, Sort 3)), `(Coup(Snd(`(⊤, Sort 3))),
Sort 2))))))))⟩))))))) &(*)) &(Sort 2)) &((Λ(⊤, `(⊤, Sort 2), Snd(`(⊤, Sort 2))) *)))
&(Sort 1)) Sort 0)
```

☑ [Formalized: `Tag k-lean-comp-type-check: Dcc.Compile.Examples`]

References

[1] Thorsten Altenkirch et al. "The Münchhausen Method in Type Theory". In: *28th International Conference on Types for Proofs and Programs (TYPES 2022)*. Ed. by Delia Kesner and Pierre-Marie Pédrot. Vol. 269. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023, 10:1–10:20. ISBN: 978-3-95977-285-3. DOI: 10.4230/LIPIcs.TYPES.2022.10. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.TYPES.2022.10.

[2] Henk Barendregt. "Introduction to generalized type systems". In: *Journal of Functional Programming* 1.2 (1991), pp. 125–154. DOI: 10.1017/S0956796800020025.

[3] G. Cousineau, P-L. Curien, and M. Mauny. "The categorical abstract machine". In: *Functional Programming Languages and Computer Architecture*. Ed. by Jean-Pierre Jouannaud. Berlin, Heidelberg: Springer Berlin Heidelberg, 1985, pp. 50–64. ISBN: 978-3-540-39677-2.

[4] P.-L. Curien. "Categorical combinators". In: *Information and Control* 69.1 (1986), pp. 188–254. ISSN: 0019-9958. DOI: https://doi.org/10.1016/S0019-9958(86)80047-X. URL: https://www.sciencedirect.com/science/article/pii/S001999588680047X.

[5] Martin Hofmann. "Extensional concepts in intensional type theory". PhD thesis. University of Edinburgh Edinburgh, UK, 1995.

[6] leanprover-community. *mathlib4*. GitHub. 2021. URL: https://leanprover-community.github.io/mathlib4_docs/Mathlib/Data/Sigma/Basic.html#Sigma.curry.

[7] Maxwell Herman Alexander Newman. "On theories with a combinatorial definition of" equivalence"". In: *Annals of mathematics* 43.2 (1942), pp. 223–243.

[8] R. A. G. Seely. "Locally cartesian closed categories and type theory". In: *Mathematical Proceedings of the Cambridge Philosophical Society* 95.1 (1984), pp. 33–48. DOI: 10.1017/S0305004100061284.